

Understanding Diffusion

How to Read These Slides

Note that in presentation mode, not all content is necessarily visible. Please also check the **hidden notes section** in the PowerPoint format, to see if there are any additional URL references related to the visible content of any particular slide. The broader references are also available in the **References** slide at the very end. However, in the hidden notes you will see more concrete pointers, such as concrete PDF pages, exact Figure numbers within papers, etc.

Some of the slides are not mandatory for conceptual understanding, but very useful if you are more mathematically inclined or simply interested in a deeper understanding. They will be depicted using a **cherry** on top (-right) **symbol** 🍒 or sometimes as a double-dose of cherry 🍒🍒 for even deeper derivations.

If you find any mistakes on the slides, I would be very grateful 😊 if you could send me an **email** at danu.caus@hereon.de



Enjoy!

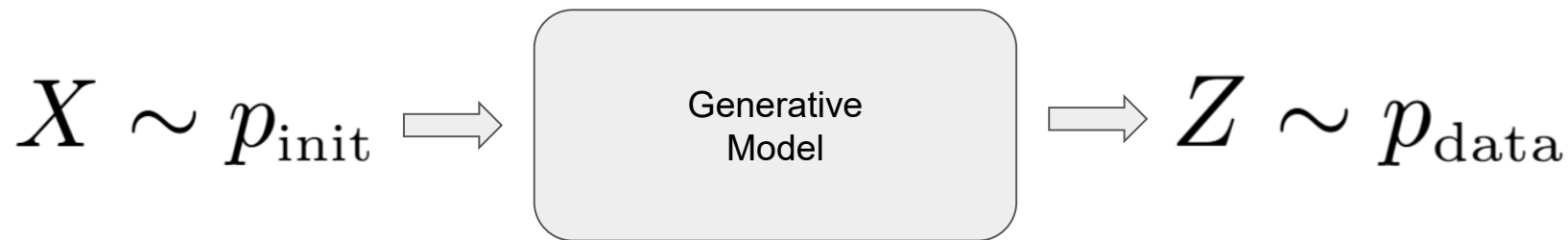


Chapter 1

**Defining Flow and Diffusion Models Through the Lens of
Differential Equations**

Flow Models

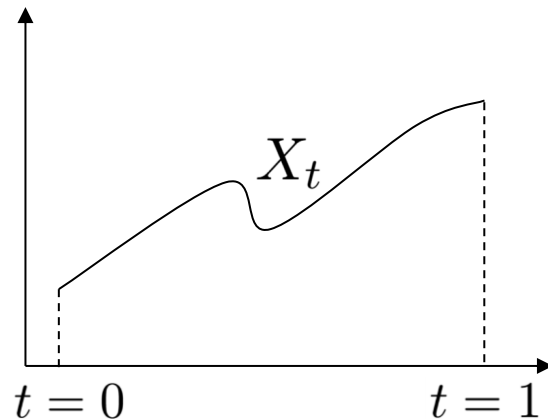
From Initial Distribution to Data Distribution



$$p_{\text{init}} = \mathcal{N}(0, I_d)$$

A Trajectory Maps Time to State

$$\begin{array}{ccc} X : [0, 1] & \longrightarrow & \mathbb{R}^d \\ \downarrow & & \downarrow \\ t & \longmapsto & X_t \end{array}$$



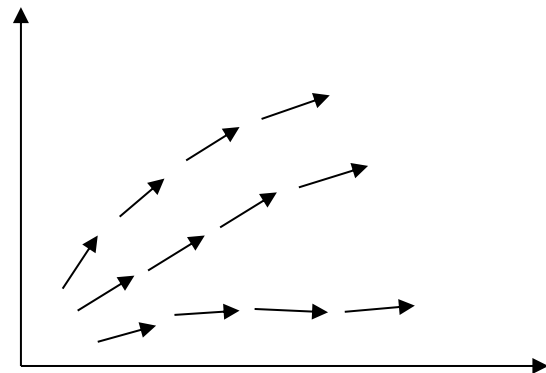
We assign each time t a vector X_t

A Vector Field Assigns a Velocity

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$$



$$(x, t) \mapsto u_t(x)$$



A vector field assigns each pair (x, t) a direction $u_t(x)$.

In other words, you could think of $u_t(x)$ as the speed vector.

Ordinary Differential Equation (ODE)

1) Initial Condition: $X_0 = x_0$ i.e. where should the trajectory start

2) How does the trajectory evolve: $\frac{d}{dt}X_t = u_t(X_t)$

One thing to note is that in the future slides you will also encounter the equivalent formula after multiplying both sides by dt :

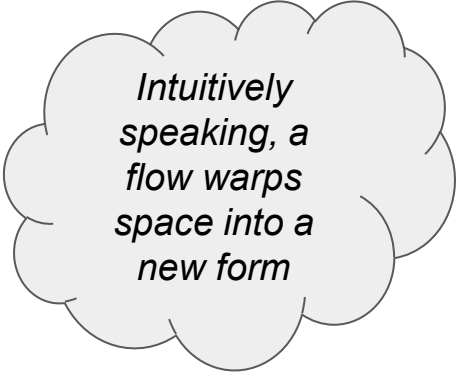
$$dX_t = u_t(X_t) dt$$

Side remark → The next state is dependent only on the current state => it is a **Markov Process**, i.e. with no memory.

A Flow Collects All ODE Trajectories

The flow corresponds to the bundle of trajectories of the ODE for all initial conditions

$$\begin{array}{ccc} \psi : \mathbb{R}^d \times [0, 1] & \rightarrow & \mathbb{R}^d \\ \Downarrow & & \Downarrow \\ (x_0, t) & \mapsto & \psi_t(x_0) \end{array}$$

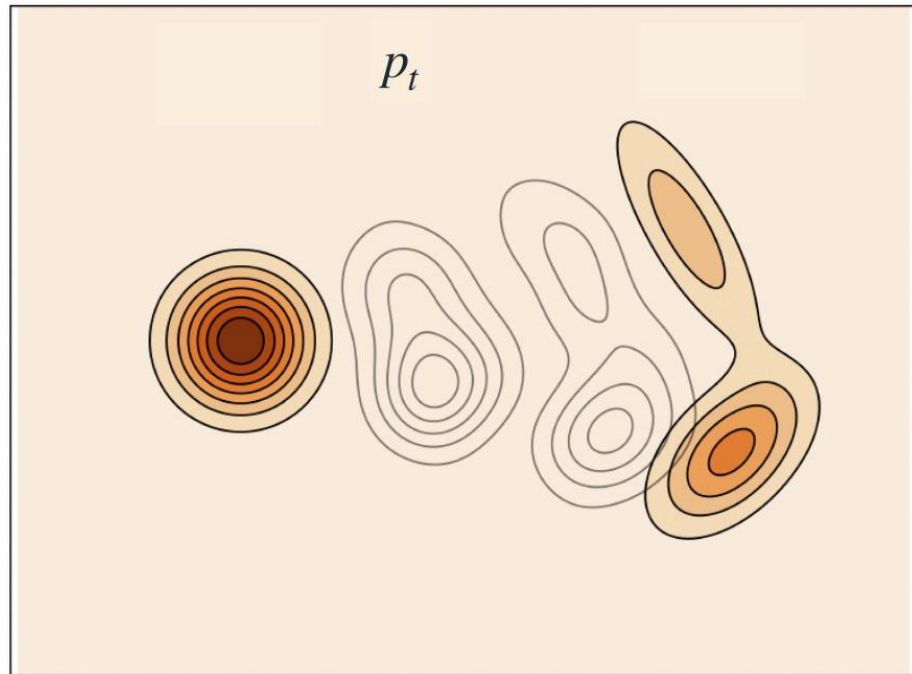
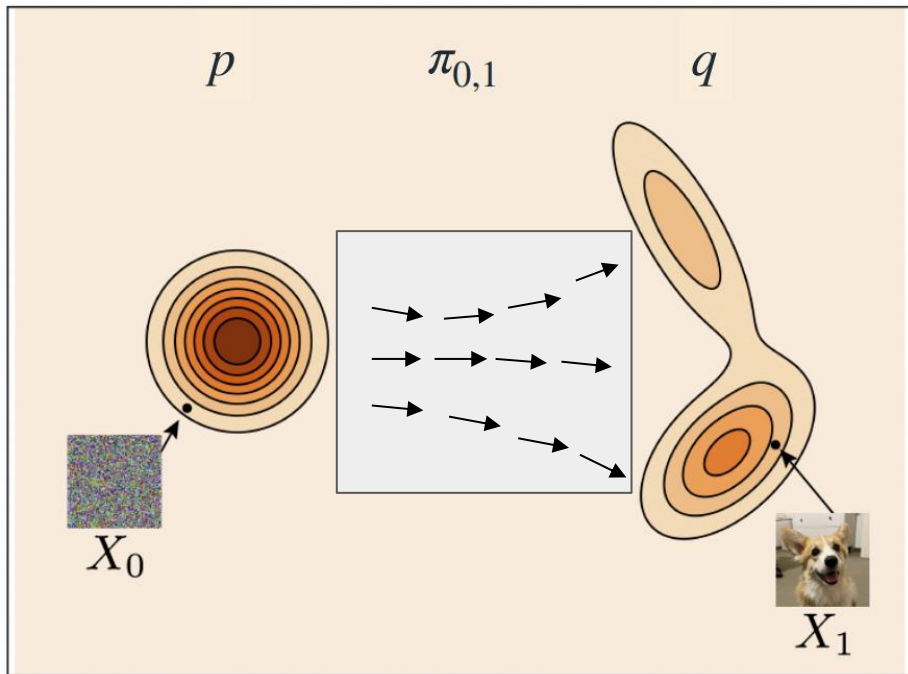


*Intuitively
speaking, a
flow warps
space into a
new form*

- 1) The trajectory at time 0 is just: $\psi_0(x_0) = x_0$
- 2) At any other time $t \neq 0$ we have the evolution form $\frac{d}{dt}\psi_t(x_0) = u_t(\psi_t(x_0))$

To gain intuition, check out this visualization tool → <https://mariogemoll.com/flow-matching> 9

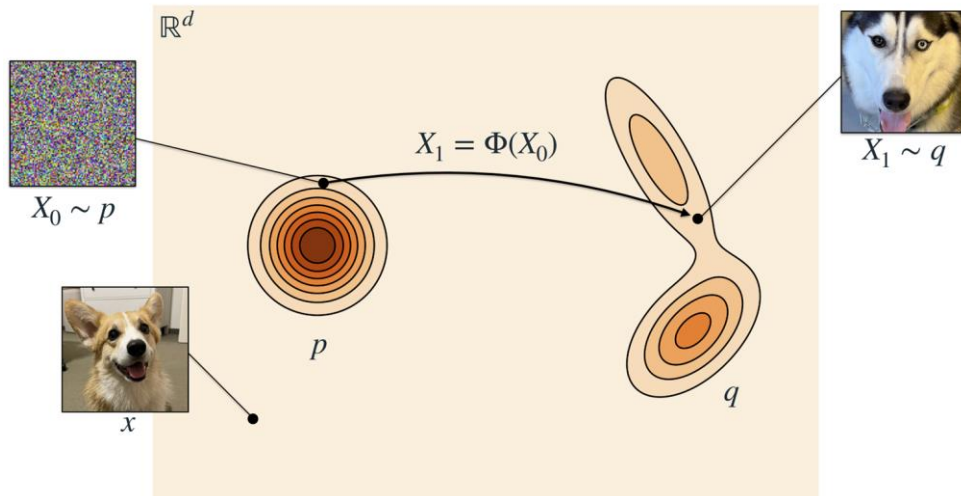
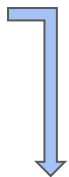
A Flow Transports Probability Density



A flow does not only move individual points. It moves an entire probability density over time: $p \rightarrow p_t \rightarrow q$. The vector field tells each point where to move. If all points follow this vector field, the whole distribution is continuously transformed from the initial distribution p into the target distribution q .

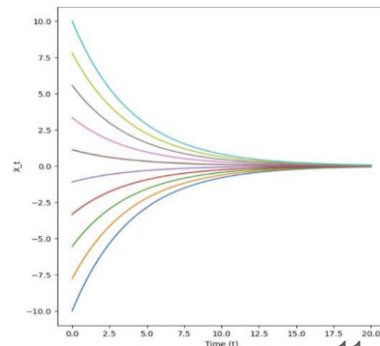
Example: A Linear Contracting Flow

Keeping in mind the intuition that a flow model changes a Probability Density Function (PDF) across time $\rightarrow$ let's try to solve this simplified example below



Say we have a Linear ODE case: $u_t(x) = -\theta x$

We claim that the flow is given by: $\psi_t(x_0) = e^{-\theta t} x_0$



Verifying the Linear Flow Solution



Proof:

1) Initial condition: $\psi_{t=0}(x_0) = e^{-\theta \cdot 0} x_0 = e^0 x_0 = x_0$

1) ODE: $\frac{d}{dt} \psi_t(x_0) = \frac{d}{dt} [e^{-\theta t} x_0] = x_0 \cdot \frac{d}{dt} e^{-\theta t} =$

Does not depend on time t so we take it out of derivative operator

This is a composed function so we have to take the inner derivative with respect to time:

$$\frac{d}{dt}(-\theta t) = -\theta$$

and then multiply with the outer derivative with respect to time:

$$\frac{d}{d\alpha} e^\alpha = e^\alpha, \text{ where } \alpha = -\theta t$$

$$= x_0 \cdot [-\theta \cdot e^{-\theta t}] = -\theta \cdot \underbrace{e^{-\theta t} x_0}_{\psi_t(x_0)} = *$$

Continuation from bottom right:

$$* = \underbrace{-\theta \psi_t(x_0)}_{u_t(\psi_t(x_0))} = u_t(\psi_t(x_0))$$

This is just $u_t(x) = -\theta x$ if we substitute the generic x with $\psi_t(x_0)$

Euler Simulation of an ODE

We use the so called “Euler Update” method to simulate the ODE as follows:

Algorithm 1 Euler update

1: Set $t = 0$

2: Set step size $h = \frac{1}{n}$

3: Set $X_0 = x_0$

4: **for** $i = 0, \dots, n - 1$ **do**

5: $X_{t+h} = X_t + h u_t(X_t)$ 

6: Update $t \leftarrow t + h$

7: **end for**

8: **return** $X_0, X_h, X_{2h}, \dots, X_1$ 

We take a small step in the direction of the vector field

Return the full trajectory from time $t = 0$ to time $t = 1$

Neural Network as a Parameterized Vector Field



However, we model $u_t(x)$ using a neural network u_t^θ , where θ are the parameters / weights of the neural network:

$$u_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad \theta \in \mathbb{R}^k$$

Hence, a neural network for our intents and purposes is literally just a parameterized vector field.

Vector Field vs. Flow

Intuitively, the neural network gives you the velocity vector, i.e. the change you need to do at a specific time and place:

$$X_{t+h} = X_t + hu_t(X_t)$$

where h is the small step in the direction of the vector field.

Note → the neural net itself is the **Vector Field**. The **Flow** on the other hand is the map that we implicitly get by simulating the ODE.

From Random Initialization to Data Distribution

We make the initial condition random: $X_{t=0} \sim p_{\text{init}} = \mathcal{N}(0, I_d)$

Follow the vector field: $\frac{d}{dt}X_t = u_t(X_t)$

Until we reach our final goal: $X_{t=1} \sim p_{\text{data}}$



Such that the end point has the data distribution we want to model.

Diffusion Models

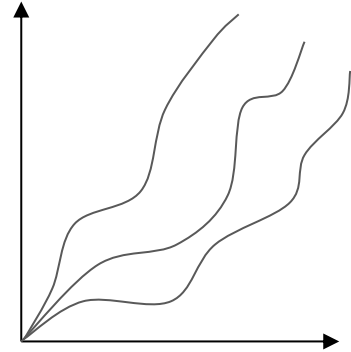
Stochastic Process

A stochastic process is a collection of random variables $\{X_t\}$ indexed by time. One realization is a trajectory or sample path.

$$X : [0, 1] \rightarrow \mathbb{R}^d$$



$$X : t \mapsto X_t$$



The collection of these trajectories is the stochastic process, i.e. different realizations produce different sample paths.

Hence, we have a trajectory that maps time t to vector X_t as previously seen in the Flow case, however this time we add the aspect of stochasticity as seen in the plot above.

Drift and Diffusion Coefficient

The vector field u is similar to what it was before and can be thought of as deterministic **drift**:

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$$

but additionally we now add a so called **diffusion coefficient** σ that adds non-determinism and controls the stochasticity:

$$\begin{array}{ccc} \sigma : [0, 1] & \rightarrow & \mathbb{R}_{\geq 0} \\ \Downarrow & & \Downarrow \\ t & \mapsto & \sigma_t \end{array}$$

Stochastic Differential Equation (SDE)

1) Initial condition: $X_0 = x_0$

2) Evolution SDE: $dX_t = \underbrace{u_t(X_t) dt}_{\text{ODE part}} + \underbrace{\sigma_t dW_t}_{\text{Added stochastic noise}}$

Brownian Motion /
Wiener Process, hence the
 W notation

Brownian Motion

Brownian motion is a stochastic process denoted as W : $W = (W_t)_{t \geq 0}$, $W_t \in \mathbb{R}^d$ such that:

At time zero: $W_0 = 0$

2) It has Gaussian increments: $W_t - W_s \sim \mathcal{N}(0, (t - s)I_d)$, where $0 \leq s < t$
i.e., the variance increases linearly in time.

3) It has independent increments, i.e.: $W_{t_1} - W_{t_0}, \dots, W_{t_n} - W_{t_{n-1}}$ are stochastically independent random variables ($0 \leq t_0 < t_1 < \dots < t_n$).

*Side note: Brownian motion is a **Gaussian Process** with a specific type of kernel.*

Why SDEs Use Increments

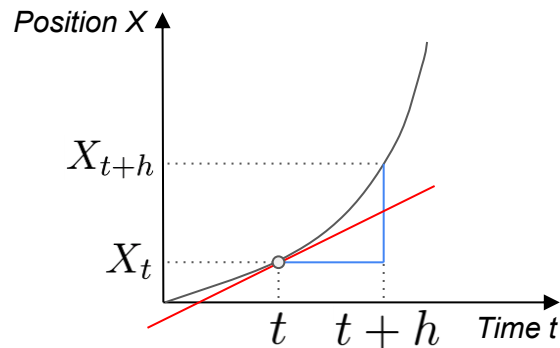
Note that the Brownian motion is **continuous**, **but** almost surely **nowhere differentiable**.
Hence, we need to describe the differential equations in a form that **does not require derivatives**.

For simplicity, let's consider the ODE case again:

$$\frac{d}{dt} X_t = u_t(X_t)$$

Mathematically this is equivalent to:

$$\lim_{h \rightarrow 0} \frac{X_{t+h} - X_t}{h} = u_t(X_t)$$



i.e., take the tangent at time t

Finite-Difference Form of an ODE



For a finite step size h , we can write the finite-difference approximation with a remainder term $R_t(h)$, where $R_t(h) \rightarrow 0$ as $h \rightarrow 0$. Hence, removing previous **lim**, we get the following equivalent equation:

$$\frac{X_{t+h} - X_t}{h} = u_t(X_t) + \underbrace{R_t(h)}_{\text{error term}}$$

Multiply both sides by $h \rightarrow X_{t+h} - X_t = hu_t(X_t) + hR_t(h)$

Add X_t to both sides $\rightarrow X_{t+h} = X_t + hu_t(X_t) + hR_t(h) \quad (*)$

From ODE Increments to SDE Increments



Transitioning to the SDE case, the $(*)$ equation will be similar here, but we would have to add the stochastic increment coming from the brownian / diffusion noise.

$$dX_t = u_t(X_t) dt + \sigma_t dW_t$$



$$X_{t+h} = X_t + \underbrace{hu_t(X_t) + hR_t(h)}_{\text{same as } (*)} + \sigma_t (W_{t+h} - W_t)$$

Because the Brownian increment is stochastic, convergence is commonly measured in mean square:

$\lim_{h \rightarrow 0} \sqrt{\mathbb{E}[\|R_t(h)\|^2]} = 0$ instead of simply: $\lim_{h \rightarrow 0} R_t(h) = 0$, as we had in the ODE case.

Reparameterization Trick

$$(W_{t+h} - W_t) \sim \mathcal{N}(0, h \cdot I_d)$$

To sample from this  is to sample from this 

However, we make use of a trick in statistics where sampling from $\mathcal{N}(0, h \cdot I_d)$ is equivalent to sampling from a **Standard Normal Distribution**, i.e. $\varepsilon \sim \mathcal{N}(0, I_d)$ and then scaling epsilon to $\sqrt{h} \cdot \varepsilon$ will ensure we're following $\mathcal{N}(0, h \cdot I_d)$

So, if $\varepsilon \sim \mathcal{N}(0, I_d)$, then $\sqrt{h} \cdot \varepsilon \sim \mathcal{N}(0, h \cdot I_d)$

More generally, if we want X to follow $\mathcal{N}(\mu, \sigma^2 \cdot I)$, we can set it to $X = \mu + \sigma \cdot \varepsilon$

This is called the reparameterization trick and is also famously used in Variational Autoencoders / VAEs

Euler–Maruyama Simulation of an SDE

Algorithm 2 Euler–Maruyama sampling scheme

- 1: Set $t = 0$
- 2: Set step size $h = \frac{1}{n}$
- 3: Set $X_0 = x_0$
- 4: **for** $i = 0, \dots, n - 1$ **do**
- 5: Draw a sample $\varepsilon \sim \mathcal{N}(0, I_d)$
- 6: $X_{t+h} = X_t + hu_t(X_t) + \sigma_t \sqrt{h} \varepsilon$
- 7: Update $t \leftarrow t + h$
- 8: **end for**
- 9: **return** $X_0, X_h, X_{2h}, X_{3h}, \dots, X_1$

Add additional noise with variance h , using our reparameterization trick, and moreover $\rightarrow$ scale it by the diffusion coefficient σ_t as we've seen in the SDE equation.

Diffusion Model as an SDE

A diffusion model converts p_{init} to p_{data} via an SDE (instead of an ODE, like in the case of flow models)



As before, we will have a neural network acting like the vector field:

$$u_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad \theta \in \mathbb{R}^k$$

Unlike in the flow case, here we introduce the diffusion coefficient σ_t

Role of the Diffusion Coefficient

Most of the time σ_t is prescribed rather than learned. It may be constant $\sigma_t = \sigma$ or follow a time-dependent schedule.

1. As before, we initialize the model with $X_0 \sim p_{\text{init}}$

Note that if sigma is zero, we cancel this term completely and thus revert to the ODE / flow model case

2. And then follow $dX_t = u_t^\theta(X_t) dt + \overset{\uparrow}{\sigma_t} dW_t$
 $\downarrow$

If sigma is on the larger side, then we will explore more. But if it's too large, we might run into numerical instability issues.

3. Until we reach our goal $\rightarrow X_1 \sim p_{\text{data}}$



If you come from a Gaussian Process machine learning background, the Brownian motion can be viewed as a *Gaussian Process* (GP), as mentioned in an earlier slide. We just use a specific type of *kernel* function, namely the Wiener Kernel:

$$k(s, t) = \min(s, t)$$

Similar to what *Gaussian distributions* are for continuous functions, the *Brownian motion* would be something equivalent, but for *stochastic processes* that are well behaved, i.e. they may not have derivatives, but they are still continuous.

All of this theory generally relates to *Markov Processes*, which is something more general, where we can also deal with “*jump processes*”, i.e. having *jump discontinuities*, which Brownian motion does not consider.

Chapter 2

**From Conditional to Marginal:
Single Data Points to the Full Distribution**

Conditional and Marginal Vector Fields

The Missing Training Target

Typically, we train the model by minimizing a **squared-error loss**:

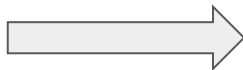
$$L(\theta; t, x) = \left\| u_t^\theta(x) - u_t^{\text{target}}(x) \right\|^2$$

In regression or classification, the training target is the label.

Here however we have no label => we have to derive a training target.

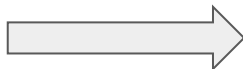
Terminology

Marginal Vector Field



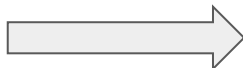
Flow Matching

Marginal Score Function



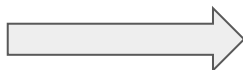
Score Matching

Conditional $\langle \textit{term} \rangle$



Per single data point

Marginal $\langle \textit{term} \rangle$



Across a distribution of data points

The 6 Objects to Remember

Tractable

Intractable integral over entire data distribution => $p_t(x)$ can't be evaluated explicitly

Conditional Probability Path

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$

Marginal Probability Path

$$p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z) dz$$

Conditional Vector Field

$$u_t^{\text{target}}(x \mid z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$$

Marginal Vector Field

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz$$

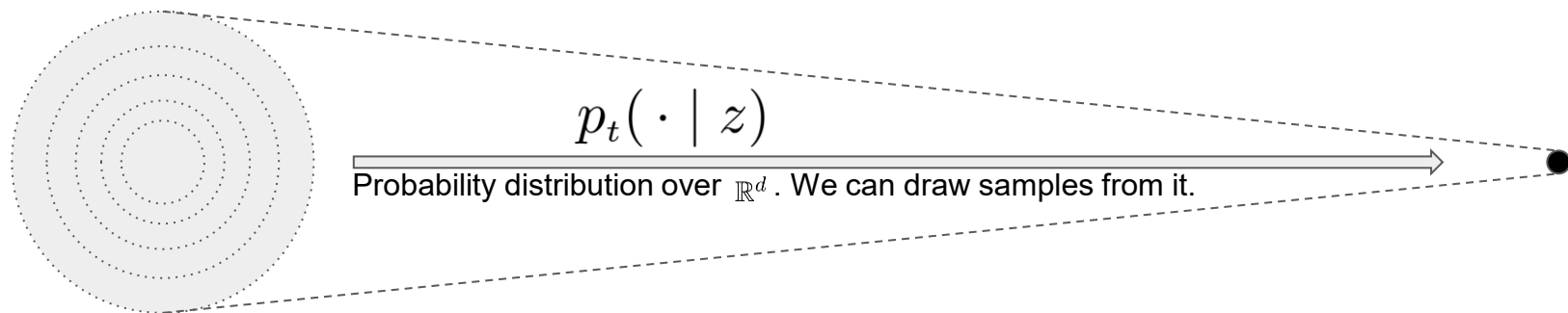
Conditional Score Function

$$\nabla \log p_t(x \mid z) = -\frac{x - \alpha_t z}{\beta_t^2}$$

Marginal Score Function

$$\nabla \log p_t(x) = \int \nabla \log p_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz$$

The Conditional Setup: Noise to a Single Data Point



Scattered / Blurry Gaussian

$$p_0(\cdot | z) = p_{\text{init}}$$

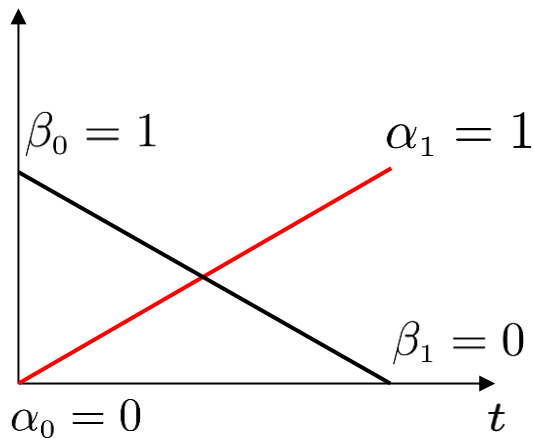
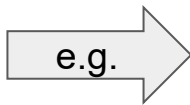
Focused dot / data point =>
can be viewed as a Dirac
Distribution $p_1(\cdot | z) = \delta_z$

Gaussian Conditional Probability Path

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$

Where α_t and β_t are called noise schedulers, which are just functions that have the following properties:

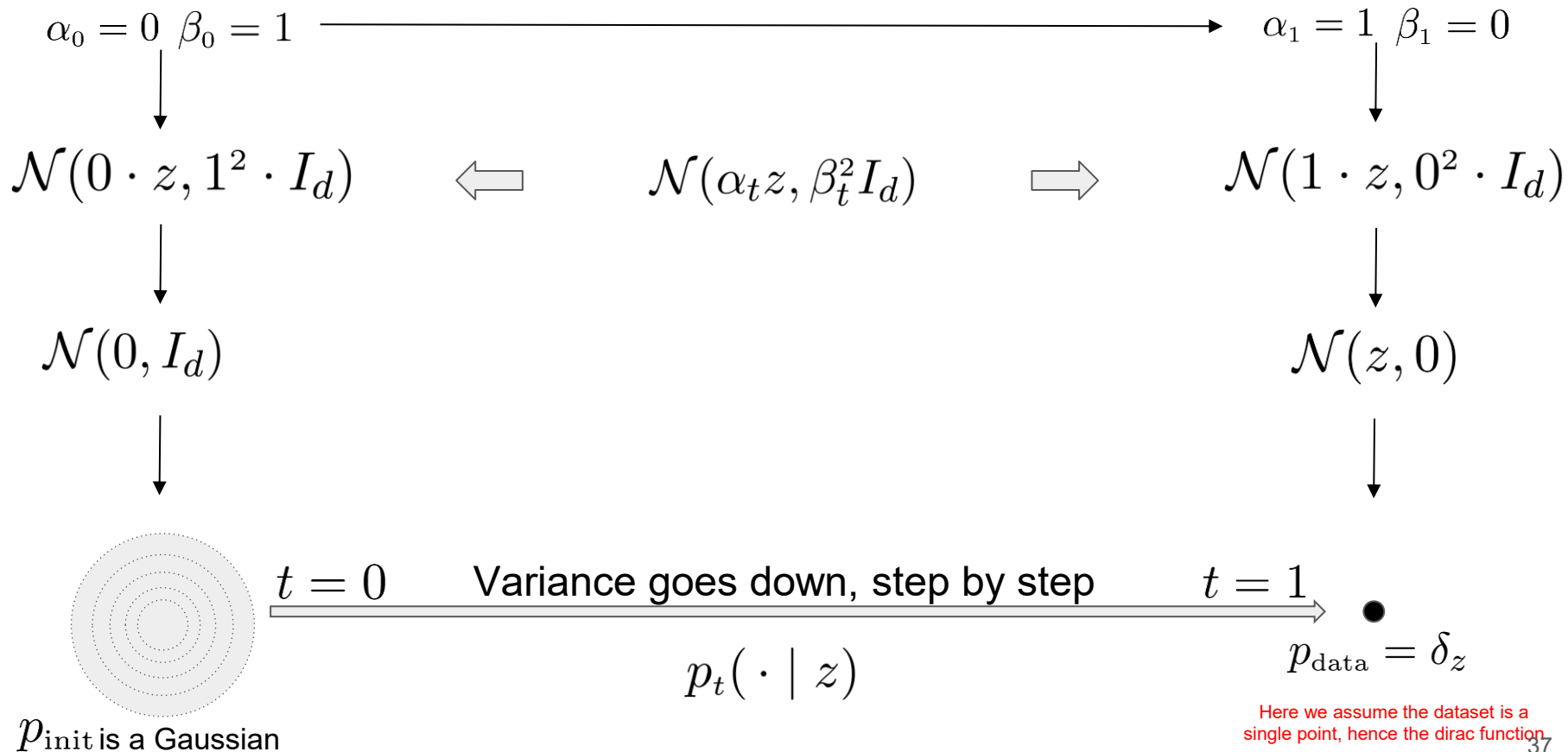
- 1) $\alpha_0 = 0$ and $\alpha_1 = 1$
- 2) $\beta_0 = 1$ and $\beta_1 = 0$



Most common example:

- 1) $\alpha_t = t$
- 2) $\beta_t = 1 - t$

Gaussian Path from Noise to a Data Point



Marginal Probability Path

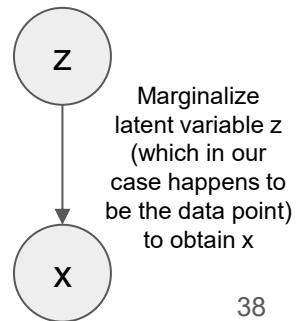
Marginalizing means **integrating** over z :

Density of z under the data distribution

$$p_t(x) = \int \underbrace{p_t(x | z)}_{\text{Conditional Probability Path}} \underbrace{p_{\text{data}}(z)}_{\text{Density of } z \text{ under the data distribution}} \boxed{dz} \Rightarrow \text{We integrate over } z$$

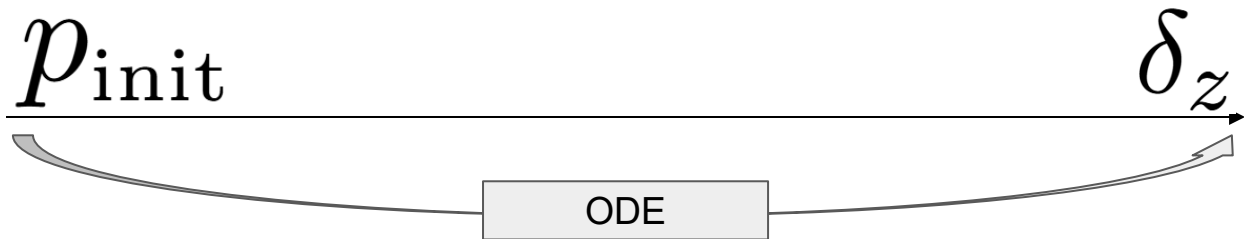
Conditional Probability Path

Every conditional path induces a marginal path. From a data distribution $p_{\text{data}}(z)$ and a conditional path $p_t(x | z)$ we get a marginal path $p_t(x)$.



A Conditional Vector Field Generates a Conditional Path

We denote the conditional vector field by: $u_t^{\text{target}}(x \mid z)$



If we sample $X_0 \sim p_{\text{init}}$ and we simulate the ODE $\frac{d}{dt}X_t = u_t^{\text{target}}(X_t \mid z)$
=> the distribution of X_t for any time between 0 and 1 is given by the
conditional path: $X_t \sim p_t(\cdot \mid z)$

Conditional Gaussian Vector Field

$$u_t^{\text{target}}(x \mid z) = \underbrace{\left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right)}_{\text{Scalar } s_1} z + \underbrace{\frac{\dot{\beta}_t}{\beta_t} x}_{\text{Scalar } s_2}$$

Where:

$$\dot{\alpha}_t = \frac{d}{dt} \alpha_t$$

$$\dot{\beta}_t = \frac{d}{dt} \beta_t$$

Since we pick α_t and β_t , at the end of the day we have just a scalar combination of z and x :

$$u_t^{\text{target}}(x \mid z) = s_1 z + s_2 x$$

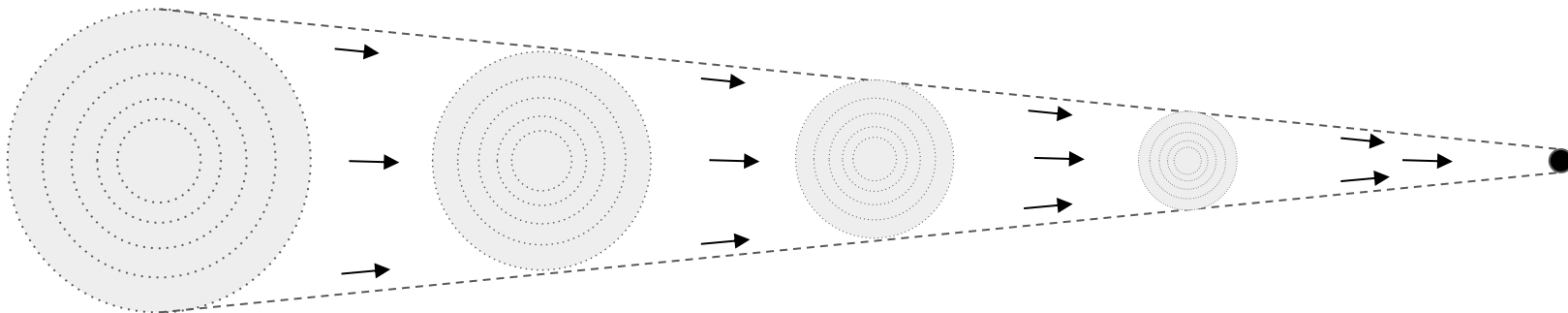
The Gaussian Vector Field Generates the Probability Path

So, if we simulate an ODE where we use the **Gaussian** Vector Field (i.e. the black arrows in the figure):

$$u_t^{\text{target}}(x \mid z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$$

we end up following the **Gaussian** Probability Path (the concentric circles in the figure):

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$



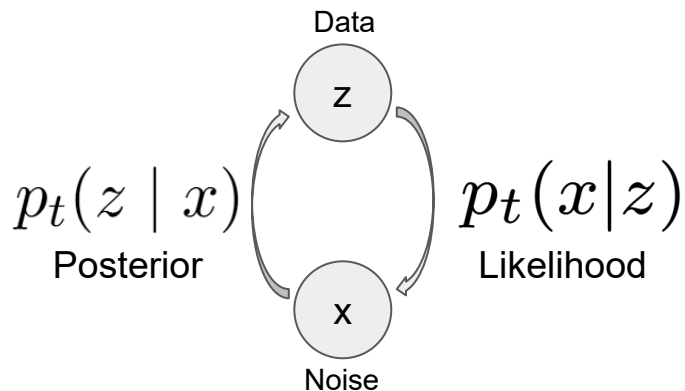
Marginalizing the Conditional Vector Fields

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \underbrace{\frac{\overbrace{p_t(x|z)}^{\text{Likelihood}} \overbrace{p_{\text{data}}(z)}^{\text{Prior}}}{p_t(x)}}_{\text{Posterior}} dz$$

Acts as a **weighting term** for the posterior

→ $p_t(z | x)$

i.e. the ratio above is equal to the posterior over Z



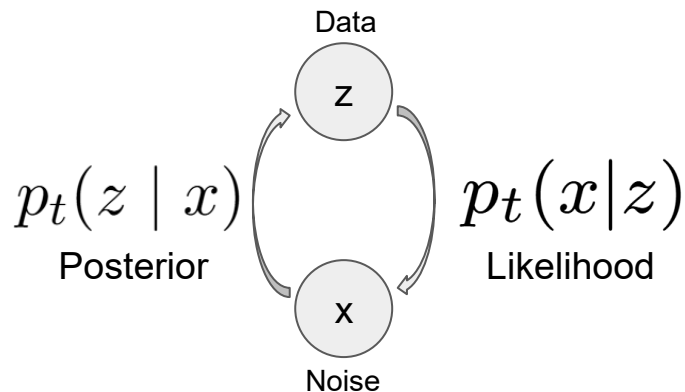
*Note that marginalizing over the posterior here is referred to as the “**Marginalization Trick**”*

Marginal Vector Field as a Posterior Expectation

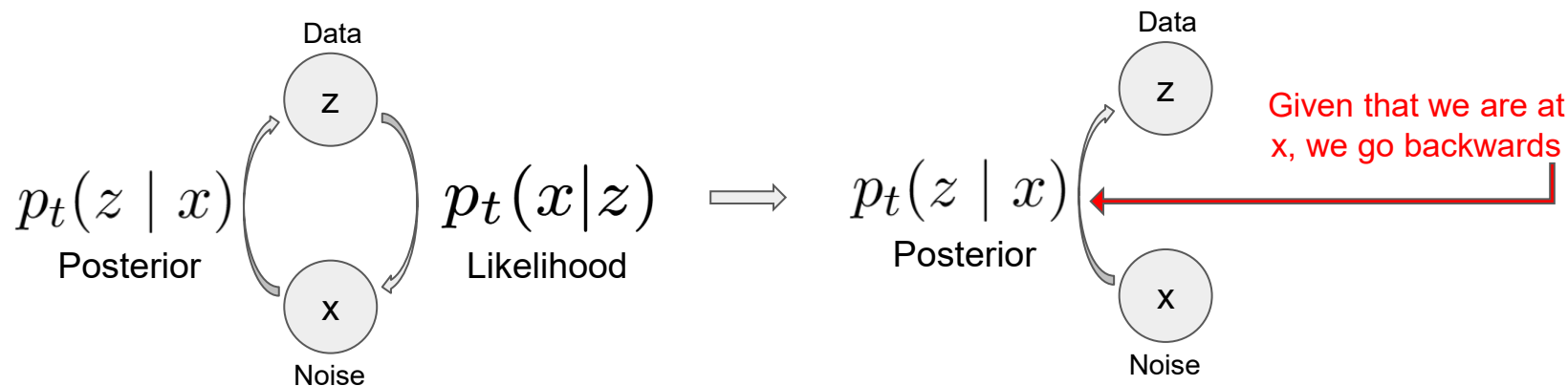
$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x \mid z) \underbrace{\frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)}}_{\text{Posterior}} dz$$

Intuitively, we can say that the marginal vector field is the weighted average over all the data points z of the conditional vector fields $u_t^{\text{target}}(x \mid z)$ where the weight is the posterior $p_t(z \mid x)$:

$$p_t(z \mid x) = \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)}$$



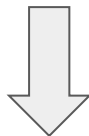
Posterior Interpretation



Given that we are at $\mathbf{x}$, we're taking the expected value (i.e. *Integral* means *Expectation*) of my conditional vector field $u_t^{\text{target}}(x | z)$. In other words, we're going backwards from $\mathbf{x}$ to $\mathbf{z}$ and taking solutions that we've used before for single data points $u_t^{\text{target}}(x | z)$, and weigh them with the posterior $p_t(z | x)$. At every time, being at $\mathbf{x}$, we're basically going backwards to our **latent** variable, which **happens to be a data point $\mathbf{z}$** and marginalizing over that.

Takeaway for the Practical Session

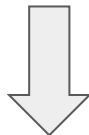
We are indirectly solving the collective problem of ending up with the data distribution, by focusing on generating single data points. In other words: generating single data points is a proxy tool of actually ending up with the overall data distribution p_{data}



The core principle is that solving the problem per data point will be enough for training the model in practice.

The Many Roads from Noise to Data

Note that there are many ways of going from noise to data. There are **infinitely many ways** actually! At the beginning people were doing this without specifying intermediate paths $X_t \sim p_t$



One key idea of diffusion models was to specify the way of going from noise to data **using intermediate paths with scalable training**.



Continuity Equation

Given a randomly initialized ODE: $X_0 \sim p_{\text{init}}, \frac{d}{dt}X_t = u_t(X_t)$

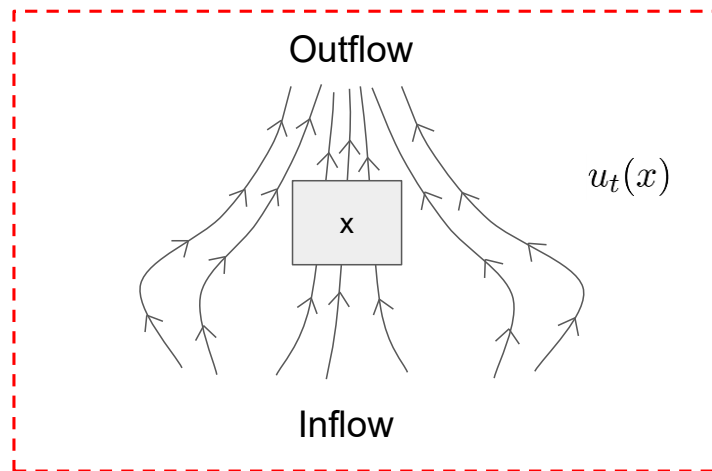
Following the probability path $X_t \sim p_t, 0 \leq t \leq 1$ is equivalent to holding the

Continuity Equation :

$$X_t \sim p_t \quad \Longleftrightarrow \quad \frac{\partial}{\partial t} p_t(x) = -\operatorname{div}(p_t u_t)(x) \quad \text{for } t \in [0, 1]$$

Where by definition $\rightarrow \operatorname{div}(u_t)(x) = \nabla \cdot u_t(x) = \sum_{i=1}^d \frac{\partial u_{t,i}(x)}{\partial x_i}, \quad u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$

Probability Mass: Inflow and Outflow



Probability-flow intuition

$$X_t \sim p_t$$

for $t \in [0, 1]$

$\Longleftrightarrow$
Importantly,
this statement
goes both ways

$$\frac{\partial}{\partial t} p_t(x) = -\operatorname{div}(p_t u_t)(x)$$

Change of
probability mass at x

Net inflow of probability mass:
Inflow - Outflow



Let's prove that $X_t \sim p_t$ by using the left part " $\leq$ " part of our equivalence relation " $\Leftrightarrow$ " from before.

We will show that we can go from $\frac{\partial}{\partial t} p_t(x)$ to $-\operatorname{div}(p_t \cdot u_t^{\text{target}})(x)$ and once this is done, we will make use of the " $\Leftrightarrow$ " relation to conclude that $X_t \sim p_t$

We end up following the probability path when we “go with the flow”

Proof: The Marginal Vector Field Generates the Marginal Path

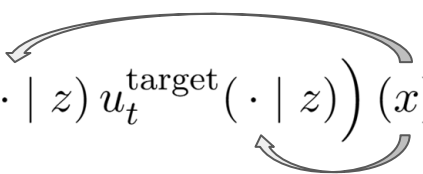


$$\frac{\partial}{\partial t} p_t(x) = \frac{\partial}{\partial t} \int p_t(x | z) p_{\text{data}}(z) dz$$

Apply the marginalization formula

$$= \int \frac{\partial}{\partial t} p_t(x | z) p_{\text{data}}(z) dz$$

Move the time derivative inside the integral

$$= - \int \text{div} \left(p_t(\cdot | z) u_t^{\text{target}}(\cdot | z) \right) (x) p_{\text{data}}(z) dz$$


Apply the continuity equation formula by using the conditional probability, i.e. with “| z” instead

$$= - \text{div} \left(\int p_t(x | z) u_t^{\text{target}}(x | z) p_{\text{data}}(z) dz \right)$$

Take the “div” operator outside and instantiate the dots with x as shown by the arrows above

$$= - \text{div} \left(p_t(x) \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \right)$$

Multiply and divide by $p_t(x)$ trick, so that we reach the posterior formula $p_t(z|x)$ in the red box

$$= - \text{div} \left(p_t(x) u_t^{\text{target}}(x) \right).$$

The integral is the **Marginal Vector Field** formula that we've seen some slides ago

Conditional and Marginal Score Functions

The Score Is a Log-Density Gradient

Whenever you see $\nabla \log$ in the diffusion context, think of “**Score**”.

Technically it's ∇_x but we will write ∇ for simplicity.

Similar as before, we can distinguish between the **conditional score**:

$$\nabla \log p_t(x \mid z)$$

and the **marginal score**:

$$\nabla \log p_t(x)$$

Marginal Score Function

$$\underbrace{\nabla \log p_t(x)}_{\text{Marginal score}} = \int \underbrace{\nabla \log p_t(x | z)}_{\text{Conditional score}} \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

To prove the formula for the marginal score function we will make use of the fact that: $\frac{d}{dx} \log x = \frac{1}{x}$ (assuming our log has base e, i.e. $\log ==$ “natural logarithm” / $\ln$)

If we have a composed function however, we would have $\frac{d}{dx} \log f(x) = \frac{1}{f(x)} df$ instead, according to the *chain-rule*.

Therefore: $\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)}$, where ∇ is the vector of partial-derivative operators:

$$\nabla_x = \left(\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \dots, \frac{\partial}{\partial x_d} \right)^\top$$



Proof: Marginal Score Identity

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)}$$

Apply the chain rule formula from last slide

$$= \frac{\nabla \int p_t(x | z) p_{\text{data}}(z) dz}{p_t(x)}$$

Use the marginalization trick, i.e. integrate the conditionals to get the marginal

$$= \frac{\int \nabla p_t(x | z) p_{\text{data}}(z) dz}{p_t(x)}$$

Take the integral outside of the gradient operator, because we integrate over z , but the gradient is with respect to x .

$$\nabla \log p_t(x | z) = \frac{\nabla p_t(x | z)}{p_t(x | z)}$$

reduce

$$= \int \nabla \log p_t(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

Apply the chain rule formula from last slide again, but for conditionals, i.e. “ $| z$ ” case (as suggested by the 3 red arrows)

$$\text{Therefore } \rightarrow \nabla \log p_t(x) = \int \nabla \log p_t(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

Marginal Score vs. Marginal Vector Field

Marginal Score Function formula:

$$\nabla \log p_t(x) = \int \nabla \log p_t(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

Both are integrals over conditionals, which act like weighting terms for the posterior $p(z|x)$

Marginal Vector Field formula:

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

We have this $p(z|x)$ posterior ratio in both cases

Both formulas are posterior expectations. The same posterior weights either the conditional scores or the conditional vector fields.

Gaussian Conditional Score

We can design the conditional probability path $p_t(x \mid z)$ ourselves, making its conditional score $\nabla \log p_t(x \mid z)$ analytically available.

For example, if we use our beloved **Gaussian Conditional Probability Path** $p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ then we can show analytically that the **Gaussian Score** is

$$\nabla \log p_t(x \mid z) = -\frac{x - \alpha_t z}{\beta_t^2}$$

From ODE to SDE

$$dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$$



Use old vector field



This score is essentially the correction term that we have to apply to the old ODE vector field.



Noise injection

Different choices of σ_t are theoretically possible, provided the corresponding score correction $\frac{\sigma_t^2}{2} \nabla \log p_t(X_t)$ is applied.

Injecting Noise: One Flow, Many SDEs

So a score function is essentially the correction term that we need to apply if we want to inject noise and convert an ODE to an SDE.

Training an ODE flow model will give us one solution. And by injecting noise on top, we can get a whole variety of different other models / solutions.

The anecdotal evidence is that a lot of the models in the literature are purely flow models, because that approach already does the job well. It's a luxury to have a whole diversity of SDE models that all do the same thing, but empirically you might still be interested to explore these and find the optimal noise level σ_t to get the “best” results, best here meaning the most visually pleasing qualitative results to you.

From Continuity to Fokker–Planck



The continuity equation we saw in the ODE case: $\frac{\partial}{\partial t} p_t(x) = -\operatorname{div}(p_t u_t)(x)$

becomes the Fokker–Planck equation in the SDE case:

$$\frac{\partial}{\partial t} p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x)$$

Change of
probability mass at x

Net inflow of probability mass:
Inflow - Outflow

Diffusion of probability mass

Compared with the continuity equation, the Fokker–Planck equation contains an additional diffusion term involving the Laplacian Δ . This is the same diffusion operator that appears in the heat equation.

Chapter 3

Practical Training Algorithms

Flow Matching vs. Score Matching

Flow matching is the training algorithm that uses the Marginal Vector Field concept:

$$u_t^{\text{target}}(x) \Rightarrow \text{Flow Matching}$$

Score matching is the training algorithm that uses the Marginal Score Function concept:

$$\nabla \log p_t(x) \Rightarrow \text{Score Matching}$$

Flow Matching: Learning the Marginal Vector Field

Goal: find a neural network with such parameters θ that would closely approximate $u_t^{\text{target}} : u_t^\theta \approx u_t^{\text{target}}$

The Flow Matching Loss is an **E**xpectation over squared differences:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E} \left[\left\| u_t^\theta(x) - u_t^{\text{target}}(x) \right\|^2 \right]$$

$t \sim \text{Unif}[0, 1]$	$\Leftarrow$	Time t is uniform in range $[0, 1]$
$z \sim p_{\text{data}}$	$\Leftarrow$	Draw a data point z , i.e. iterate over the data loader
$x \sim p_t(\cdot \mid z)$	$\Leftarrow$	Draw x from conditional path

Why the Flow Matching Loss Is Intractable

$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E} \left[\left\| u_t^\theta(x) - u_t^{\text{target}}(x) \right\|^2 \right]$ This loss function is the objective we want to minimize ideally. However, it is not tractable due to $u_t^{\text{target}}(x)$ not being trivial to evaluate.

$t \sim \text{Unif}[0, 1], z \sim p_{\text{data}}, x \sim p_t(\cdot | z)$

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

$p_t(z|x)$

This is a very high dimensional integral and basically intractable.

Why intractable? Well, we could technically loop over the entire dataset and for every datapoint evaluate the posterior ratio $p_t(z|x)$ and then take an average over that. But let's say our dataset has 100 million images => for every evaluation of the neural network we would need to do this. This is not realistic.

Conditional Flow Matching Loss

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E} \left[\left\| u_t^\theta(x) - u_t^{\text{target}}(x \mid z) \right\|^2 \right]$$

$$t \sim \text{Unif}[0, 1]$$

$$z \sim p_{\text{data}}$$

$$x \sim p_t(\cdot \mid z)$$



Since we have a formula for this, $\mathcal{L}_{\text{CFM}}$ will be tractable.

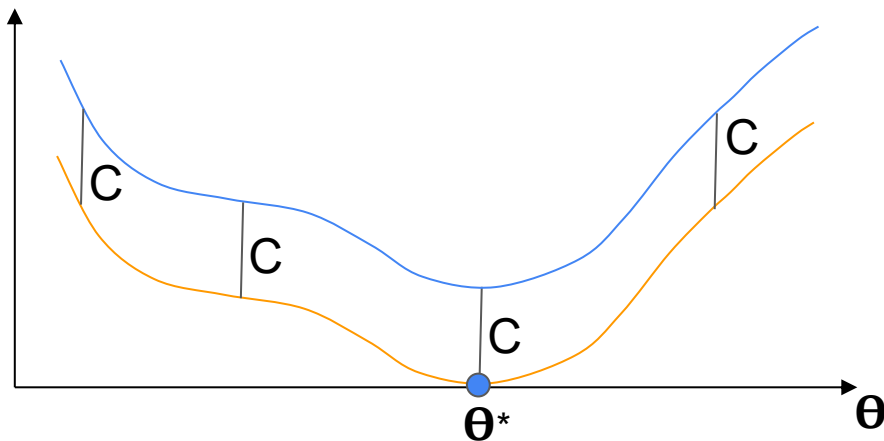
$\mathcal{L}_{\text{CFM}}$ is tractable, but does it even make sense to minimize this object? Because we don't just want to generate single data points z . We want to generate distributions of data points, which is what $u_t^{\text{target}}(x)$ is doing from the $\mathcal{L}_{\text{FM}}$ case.

Why Conditional Flow Matching Is a Valid Proxy

We will now show that we can indeed minimize L_{CFM} as a proxy for minimizing L_{FM}

Theorem: $\mathcal{L}_{\text{CFM}}(\theta) = \mathcal{L}_{\text{FM}}(\theta) + C$ for $C \geq 0$

where C is a constant independent of θ , and non-negative, i.e. L_{CFM} is shifted above L_{FM}



L_{FM} can reach zero, whereas L_{CFM} generally does not. The two losses differ by a non-negative constant C .

We don't care about the value of L_{CFM} on the y-axis actually. We only care about θ^* . And θ^* is the same for both L_{FM} and L_{CFM} .

Consequences of the Constant Offset

There are 2 important properties that follow:

1) For minimizer θ^* of $\mathcal{L}_{\text{CFM}}$: $u_t^{\theta^*} = u_t^{\text{target}}$

So if we train perfectly, we will reach this equality $\Rightarrow$ if we simulate our neural net model, we will have noise being converted into data, which was our overall goal.

2) $\nabla_{\theta} \mathcal{L}_{\text{CFM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{FM}}(\theta) + \cancel{\nabla_{\theta} C}$ Reduces to zero because the constant C does not depend on θ

$\Downarrow$

$$\nabla_{\theta} \mathcal{L}_{\text{CFM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{FM}}(\theta)$$

Hence, both losses guide the model toward the same solution. The individual mini-batch updates may differ, but their **average direction is the same**.

General Flow Matching Training Procedure

Algorithm 3 Flow Matching Training Procedure (general version)

Require: A dataset of samples $z \sim p_{\text{data}}$; neural network u_t^θ

- 1: **for** each mini-batch of data **do**
 - 2: Sample a data example z from the dataset
 - 3: Sample a random time $t \sim \text{Unif}[0, 1]$
 - 4: Sample $x \sim p_t(\cdot \mid z)$
 - 5: Compute the loss $\mathcal{L}(\theta) = \left\| u_t^\theta(x) - u_t^{\text{target}}(x \mid z) \right\|^2$
 - 6: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$
 - 7: **end for**
-

By averaging over many samples drawn by the Data Loader, the empirical mean of the conditional terms approaches the corresponding marginal expectation $u_t^{\text{target}}(x)$.

The model u_t^θ has 2 inputs: the time t and location x . *It then learns to regress towards each data point individually, and in the long run it will learn to create entire datasets.*

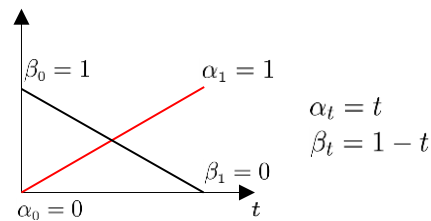
CFM Loss for a Gaussian Conditional Path

Note → we can pick the conditional probability path ourselves, and then we can pick the conditional vector field that does the job we want. This is essentially a design choice that is up to us. Hence, we can choose the Gaussian variants of p and u .

1) Choose Gaussian probability path: $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$

2) Choose Gaussian conditional vector field: $u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$

$$\text{If } \varepsilon \sim \mathcal{N}(0, I_d) \quad \implies \quad \alpha_t z + \beta_t \varepsilon = x \sim p_t(\cdot | z)$$



Gaussian CFM Loss: Derivation I



$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ x \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)}} \left[\left\| u_t^\theta(x) - \underbrace{\left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z - \frac{\dot{\beta}_t}{\beta_t} x}_{u_t^{\text{target}}(x | z) \text{ for the Gaussian } p_t(\cdot | z)} \right\|^2 \right]$$

$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}}}} \left[\left\| u_t^\theta(\alpha_t z + \beta_t \varepsilon) - \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z - \frac{\dot{\beta}_t}{\beta_t} (\alpha_t z + \beta_t \varepsilon) \right\|^2 \right]$$

$\varepsilon \sim \mathcal{N}(0, I_d)$ *Change of variable because $x = \alpha_t z + \beta_t \varepsilon$ (See previous slide)*

$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1], \\ z \sim p_{\text{data}}, \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| u_t^\theta(\alpha_t z + \beta_t \varepsilon) - \dot{\alpha}_t z + \cancel{\frac{\dot{\beta}_t \alpha_t z}{\beta_t}} - \cancel{\frac{\dot{\beta}_t \alpha_t z}{\beta_t}} - \cancel{\frac{\dot{\beta}_t \beta_t \varepsilon}{\beta_t}} \right\|^2 \right]$$

Gaussian CFM Loss: Derivation II



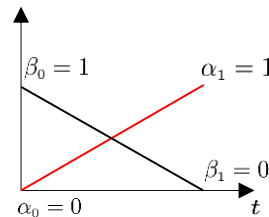
$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| u_t^\theta(\alpha_t z + \beta_t \varepsilon) - \dot{\alpha}_t z - \dot{\beta}_t \varepsilon \right\|^2 \right]$$

where $\dot{\alpha}_t = \frac{d}{dt}\alpha_t$ and $\dot{\beta}_t = \frac{d}{dt}\beta_t$

$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| u_t^\theta(\underbrace{\alpha_t z + \beta_t \varepsilon}_{\text{We plug in some weighted average of the } z \text{ data point and noise } \varepsilon \text{ and are predicting}}) - \underbrace{(\dot{\alpha}_t z + \dot{\beta}_t \varepsilon)}_{\text{an average between the } z \text{ data point and } \varepsilon\text{-noise, but using time derivative weights instead}} \right\|^2 \right]$$

Conditional OT Path

To make things even more concrete, let's choose a specific instance of α_t and β_t , say $\alpha_t = t$ and $\beta_t = 1 - t \implies$

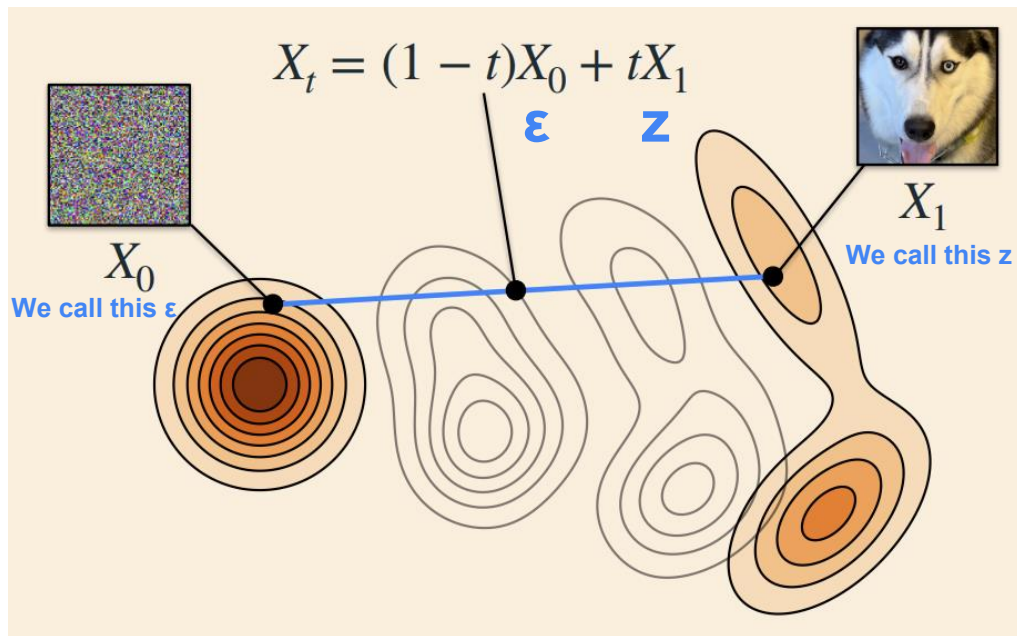


The corresponding derivatives with respect to time will be $\dot{\alpha}_t = 1$ and $\dot{\beta}_t = -1$, so:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}[0,1] \\ z \sim p_{\text{data}} \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| \underbrace{u_t^\theta(tz + (1-t)\varepsilon)}_{\substack{\text{Straight line} \\ \text{between data} \\ z \text{ and noise } \varepsilon}} - \underbrace{(z - \varepsilon)}_{\substack{\text{Difference} \\ \text{between data} \\ \text{and noise}}} \right\|^2 \right]$$

This is referred to as the **Cond OT Path** (i.e. **Conditional Optimal Transport Path**), i.e. an **optimal transport path**, essentially corresponding to a straight line between noise and data, which is $tz + (1-t)\varepsilon$ and what we are predicting is the difference $(z - \varepsilon)$

Conditional OT Path Intuition



[Slide 19 from <https://neurips.cc/media/neurips-2024/Slides/99531.pdf>]

Conditional OT Algorithm

Algorithm 4 Flow Matching for the Conditional OT Path

Require: A dataset of samples $z \sim p_{\text{data}}$; neural network u_t^θ

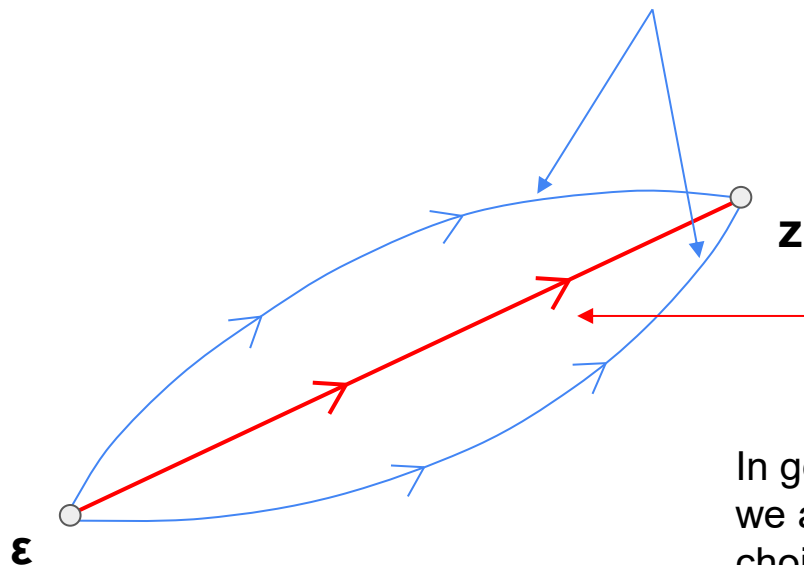
- 1: **for** each mini-batch of data **do**
 - 2: Sample a data example z from the dataset
 - 3: Sample a random time $t \sim \text{Unif}[0, 1]$
 - 4: Sample noise $\varepsilon \sim \mathcal{N}(0, I_d)$
 - 5: Set $x = tz + (1 - t)\varepsilon$
 - 6: Compute the loss $\mathcal{L}(\theta) = \left\| u_t^\theta(x) - (z - \varepsilon) \right\|^2$ 
 - 7: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$
 - 8: **end for**
-

Notice that we are learning to predict the same thing at all times t , namely $(z - \varepsilon)$, but at inference phase we have to do it sequentially, since at $t=0$ we have access to no information at all, i.e. we have just ε -chaos, and at $t = 1$ we have access to full information. Hence, we have to gain this information step-by-step, for all $t \in [0, 1]$.

So over time we learn to predict the same thing $(z - \varepsilon)$, but the amount of information grows, i.e. axis of information will still differ across time.

Physical Intuition

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E} \left[\left\| u_t^\theta (\alpha_t z + \beta_t \epsilon) - \underbrace{(\dot{\alpha}_t z + \dot{\beta}_t \epsilon)}_{\text{Changing velocity}} \right\|^2 \right] \xrightarrow{\text{Cond OT special case}} \mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E} \left[\left\| u_t^\theta (tz + (1-t)\epsilon) - \underbrace{(z - \epsilon)}_{\text{Constant velocity}} \right\|^2 \right]$$



$$\text{velocity} = \frac{\text{displacement}}{\text{time}} = \frac{z - \epsilon}{1 - 0} = z - \epsilon$$

In general, with the Conditional Flow Matching Loss CFM we are essentially predicting velocities across our path of choice. And if we take the straight **Cond OT path**, our velocity becomes constant, i.e. **(z-ε)**

Constant-Offset Identity Proof I



Because our loss functions resemble the structure $(a-b)^2$ we can make use of the binomial expansion formula, just treating **a** and **b** as vectors instead of scalars. We get the so-called **Squared Euclidean Norm Expansion** formula:

$$\|a - b\|^2 = \|a\|^2 - 2a^\top b + \|b\|^2 \quad a, b \in \mathbb{R}^d$$

Let's apply this to both losses

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| u_t^\theta(x) - u_t^{\text{target}}(x) \right\|^2 \right]$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| u_t^\theta(x) - u_t^{\text{target}}(x | z) \right\|^2 \right]$$



Constant-Offset Identity Proof II

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| u_t^\theta(x) \right\|^2 - 2 \left(u_t^\theta(x) \right)^\top u_t^{\text{target}}(x) + \left\| u_t^{\text{target}}(x) \right\|^2 \right]$$

(1) Appears in both cases

(3) Does not depend on θ

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| u_t^\theta(x) \right\|^2 - 2 \left(u_t^\theta(x) \right)^\top u_t^{\text{target}}(x | z) + \left\| u_t^{\text{target}}(x | z) \right\|^2 \right]$$

$$(2) \mathbb{E}_{t,z,x} \left[2 \left(u_t^\theta(x) \right)^\top u_t^{\text{target}}(x) \right] = \mathbb{E}_{t,z,x} \left[2 \left(u_t^\theta(x) \right)^\top u_t^{\text{target}}(x | z) \right]$$

Because first terms are the same (1), second terms are the same in expectation (2), and third terms will also differ in expectation by some constant C not depending on θ (3) $\Rightarrow$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathcal{L}_{\text{FM}}(\theta) + C$$

Flow Model Sampling

Now that we have trained the model, we will sample from it by using the algorithm we have seen before, where we plug in our newly trained vector field u_t^θ

Algorithm 5 Sampling from a Flow Model with Euler's Method

Require: Neural-network vector field u_t^θ , number of steps n

1: Set $t \leftarrow 0$

2: Set the step size $h \leftarrow \frac{1}{n}$

3: Draw a sample $X_0 \sim p_{\text{init}}$



Random initialization

4: **for** $i = 1, \dots, n$ **do**

5: $X_{t+h} \leftarrow X_t + h u_t^\theta(X_t)$

6: $t \leftarrow t + h$

7: **end for**

8: **return** X_1



Return final point

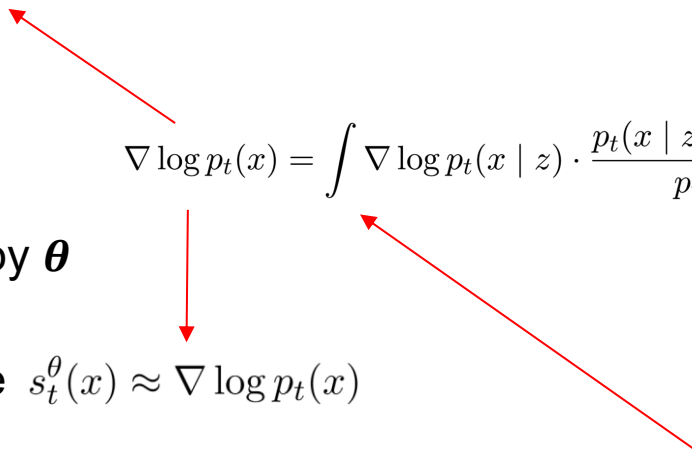
One might wonder to what extent are flow models **deterministic**?

They are in fact deterministic once you sample the initial point. So, we have a **random initialization** indeed! But past that, the **trajectory X_0 to X_1** is in fact deterministic, i.e. there is a one-to-one correspondence between the input and the output, which is given by the flow-map.

Score Matching: Learning the Marginal Score

Remember: the SDE extension trick we learned before shows how the marginal score function allows to extend the ODE to an SDE with arbitrary diffusion coefficient:

$$X_0 \sim p_{\text{init}} , \quad dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t \quad \Rightarrow \quad X_t \sim p_t$$

$$\nabla \log p_t(x) = \int \nabla \log p_t(x | z) \cdot \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$


We have a Score Network: $\mathbf{s}_t^\theta$ parametrized by θ

The goal is to approximate the marginal score $s_t^\theta(x) \approx \nabla \log p_t(x)$

As before, we will have a theoretical loss that is intractable because of the integral:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| s_t^\theta(x) - \nabla \log p_t(x) \right\|^2 \right]$$

Denoising Score Matching Loss

We would then use the tractable version of the loss instead, just as we have done previously. This could be referred to as the “Conditional Score Matching Loss”, but the formal name in the literature is “Denoising Score Matching Loss”, i.e. $\mathcal{L}_{\text{DSM}}$:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{t,z,x} \left[\left\| s_t^\theta(x) - \nabla \log p_t(x \mid z) \right\|^2 \right]$$


The $\mathcal{L}_{\text{DSM}}$ is tractable because we have an analytical formula for $\nabla \log p_t(x \mid z)$,

namely :

$$\nabla \log p_t(x \mid z) = -\frac{x - \alpha_t z}{\beta_t^2}$$

Déjà vu

Just as before the following holds: $\mathcal{L}_{\text{DSM}}(\theta) = \mathcal{L}_{\text{SM}}(\theta) + C$, $C \geq 0$

With the two similar **properties**:

1) For θ^* of $\mathcal{L}_{\text{DSM}}$: $s_t^{\theta^*}(x) = \nabla_x \log p_t(x)$

2) The gradients for the 2 losses are the same: $\nabla_{\theta} \mathcal{L}_{\text{DSM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{SM}}(\theta)$

Score Matching Training Procedure

Algorithm 6 Score-Matching Training Procedure (general version)

Require: A dataset of samples $z \sim p_{\text{data}}$, score network s_t^θ

- 1: **for** each mini-batch of data **do**
 - 2: Sample a data example z from the dataset
 - 3: Sample a random time $t \sim \text{Unif}[0, 1]$
 - 4: Sample $x \sim p_t(\cdot \mid z)$
 - 5: Compute the loss $\mathcal{L}(\theta) = \left\| s_t^\theta(x) - \nabla_x \log p_t(x \mid z) \right\|^2$
 - 6: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$
 - 7: **end for**
-

The loss here is generic, but we will get an elegant special case once we choose by design Gaussian Probability Paths.

Denoising Score Matching for a Gaussian Probability Path

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E} \left[\left\| s_t^\theta(x) - \nabla_x \log p_t(x | z) \right\|^2 \right] \quad \nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}$$

Plug in

This is our design choice

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ x \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)}} \left[\left\| s_t^\theta(x) + \frac{x - \alpha_t z}{\beta_t^2} \right\|^2 \right]$$

Apply $x = \alpha_t z + \beta_t \varepsilon \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, i.e. reparameterization trick.

$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| s_t^\theta(\alpha_t z + \beta_t \varepsilon) + \frac{\cancel{\alpha_t z} + \beta_t \varepsilon - \cancel{\alpha_t z}}{\beta_t^2} \right\|^2 \right]$$

$$= \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1], \\ z \sim p_{\text{data}}, \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| s_t^\theta(\alpha_t z + \beta_t \varepsilon) + \frac{\varepsilon}{\beta_t} \right\|^2 \right]$$

Using this **analytical formula**, we get this final **“Denoising Score Matching Loss”**. We essentially predict the noise ε that was used to corrupt the data point, hence the term “denoising” in its name.

Denoising Score Matching Interpretation

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}[0, 1] \\ z \sim p_{\text{data}} \\ \varepsilon \sim \mathcal{N}(0, I_d)}} \left[\left\| \underbrace{s_t^\theta(\alpha_t z + \beta_t \varepsilon)}_{\text{We are taking some interpolated input between data point } \mathbf{z} \text{ and noise } \boldsymbol{\varepsilon}} + \underbrace{\frac{\varepsilon}{\beta_t}}_{\text{and learn to predict negatively scaled noise } -\boldsymbol{\varepsilon} / \beta_t} \right\|^2 \right]$$

Note that this loss can be numerically unstable for low β , i.e. if β approaches zero, then the loss blows up towards infinity.

Score Matching Training Procedure for a Gaussian Probability Path

Algorithm 7 Score-Matching Training Procedure for Gaussian Probability Path

Require: Schedules α_t, β_t with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$

- 1: **for** each mini-batch of data **do**
 - 2: Sample a data example z from the dataset
 - 3: Sample a random time $t \sim \text{Unif}[0, 1]$
 - 4: Sample noise $\varepsilon \sim \mathcal{N}(0, I_d)$
 - 5: Set $x_t \leftarrow \alpha_t z + \beta_t \varepsilon$
 - 6: Compute the loss $\mathcal{L}(\theta) = \left\| s_t^\theta(x_t) + \frac{\varepsilon}{\beta_t} \right\|^2$
 - 7: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$
 - 8: **end for**
-

We get an elegant special case of our Score Matching Procedure if we choose by design Gaussian Probability Paths.

Learning the Drift and Score

Start with: $X_0 \sim p_{\text{init}}$

Exact result:
$$dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$$

Plug in neural networks

Approximate:
$$dX_t = \left[u_t^\theta(X_t) + \frac{\sigma_t^2}{2} s_t^\theta(X_t) \right] dt + \sigma_t dW_t$$

End with: After training $\Rightarrow X_t \sim p_t \Rightarrow$ and therefore $X_1 \sim p_{\text{data}}$

Denoising Diffusion Models (DDMs)

Denoising Diffusion Models = Diffusion Models with a Gaussian Probability Path:

$$\mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$

Many people drop the word “denoising” and simply use the term “diffusion model”.

The DDMs have the special property that **we can convert score matching to flow matching and vice-versa.**

The first generation of diffusion models only did score matching, because they were implicitly relying on the Gaussian probability path.

Vector Field as a Linear Combination

We've already seen that:

$$u_t^{\text{target}}(x \mid z) = \underbrace{\left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right)}_{\lambda_1} z + \underbrace{\frac{\dot{\beta}_t}{\beta_t} x}_{\lambda_2}$$

which can be re-written as:

$$u_t^{\text{target}}(x \mid z) = \lambda_1 z + \lambda_2 x$$

Score as a Linear Combination

Similarly, recall that:

$$\nabla \log p_t(x \mid z) = -\frac{x - \alpha_t z}{\beta_t^2} = \underbrace{-\frac{1}{\beta_t^2} x}_{\lambda_1} + \underbrace{\frac{\alpha_t}{\beta_t^2} z}_{\lambda_2}$$

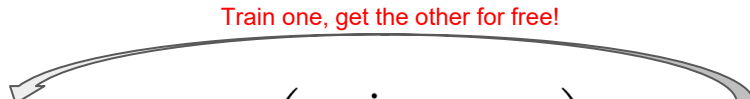
Hence, the score is also of the form:

$$\nabla \log p_t(x \mid z) = \lambda_1 z + \lambda_2 x$$

From Score Matching to Flow Matching: Conditional Case

Notice that λ_1 and λ_2 are not exactly the same in the 2 formulas we've seen, but this is not the important aspect. The main idea is that we get a **linear relationship** in both cases. Hence, we can say that both $u_t^{\text{target}}(x | z)$ and $\nabla \log p_t(x | z)$ are **weighted combinations of x and z** $\Rightarrow$ **we can convert them into each other!**

Mathematically it can be proven that:


$$u_t^{\text{target}}(x | z) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x$$

So, we can compute only the score for example, and then get the vector field **for free after training** by using the analytical formula above.

From Score Matching to Flow Matching: Marginal Case

And if we marginalize over the variable z , a similar formula holds:

$$u_t^{\text{target}}(x) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x) + \frac{\dot{\alpha}_t}{\alpha_t} x$$

The approximate equivalent of this, using neural networks parametrized by θ will be:

$$u_t^\theta(x) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) s_t^\theta(x) + \frac{\dot{\alpha}_t}{\alpha_t} x$$

Hence, we only need to train just one
neural net instead of two

Note that we use the conditional formulas in the code, but the marginal object is what implicitly gets computed over large datasets.

Time Conventions and Extensions

We use the convention that noise resides at time $t=0$ and data is at $t=1$. But in the literature you can also find the convention that data is at $t=0$ and noise at $t=1$. This is called **inverted time convention**.

Historically, diffusion models were often introduced through discrete-time Markov noising and denoising chains. Here we use the **continuous-time ODE/SDE** and **stochastic interpolant viewpoint**.

Our framework also allows us to go from **arbitrary p_{init}** to **arbitrary p_{data}**

For example, we can go:

- *From video without audio to video with audio*
- *From low resolution data to high resolution version of the same data*
- *From partially occluded scenes in robotics, to fully visible scenes, i.e. unmasking*

References

<https://diffusion.csail.mit.edu/2026/index.html>

https://diffusion.csail.mit.edu/2026/docs/lecture_notes.pdf

<https://neurips.cc/media/neurips-2024/Slides/99531.pdf>

https://github.com/facebookresearch/flow_matching

<https://mariogemoll.com/flow-matching>

Chapter 4

**Conditional Generation /
Image generation based on prompts**

Coming Soon

Classifier Guidance

Coming next Spring

Classifier-Free Guidance

Coming next Spring